



Experiment 03

Ranking, Aggregate Subqueries & Window Functions in SQL

Name: Deven

UID: 24BCF10060

Aim

To study and implement advanced SQL concepts such as correlated aggregate subqueries, the `HAVING` clause, and the `DENSE_RANK()` window function to solve real-world data analysis and reporting problems.

Question 1 (Easy)

You are given the `Employee` schema shown below; the task is to find the highest score for each respective department. For example, department D1 has the highest score as 5.28; hence in the output, the final updated score for every row belonging to D1 should be 5.28. The same logic applies to the other departments, i.e., D2, D3 and D4.

Given Data

```
1 CREATE TABLE Employee
2 (
3     eid      INT PRIMARY KEY,
4     dept     VARCHAR(10),
5     scores   NUMERIC(5,2)
6 );
7
8 INSERT INTO Employee VALUES
9 (1, 'D1', 1),
10 (2, 'D1', 5.28),
11 (3, 'D1', 4),
12 (4, 'D2', 8),
13 (5, 'D1', 2.5),
14 (6, 'D2', 7),
15 (7, 'D3', 9),
16 (8, 'D4', 10.2);
```

Answer

```
1 UPDATE Employee e
2
3 SET scores = sub.max_score
4
5 FROM
6 (
7     SELECT dept, MAX(scores) AS max_score
8     FROM Employee
9     GROUP BY dept
10 ) sub
11
12 WHERE e.dept = sub.dept;
13
14 SELECT * FROM Employee
```



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

15 `ORDER BY eid;`

Output

eid	dept	scores
1	D1	5.28
2	D1	5.28
3	D1	5.28
4	D2	8.00
5	D1	5.28
6	D2	8.00
7	D3	9.00
8	D4	10.20



Question 2 – Sales Analysis Part 1 (Medium)

A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.

Given Data

```
1 CREATE TABLE SalesPerson
2 (
3     seller_id INT PRIMARY KEY,
4     name VARCHAR(50),
5     city VARCHAR(50),
6     commission NUMERIC(5,2)
7 );
8
9 CREATE TABLE Company
10 (
11     com_id INT PRIMARY KEY,
12     name VARCHAR(50),
13     city VARCHAR(50)
14 );
15
16 CREATE TABLE Orders
17 (
18     order_id INT PRIMARY KEY,
19     seller_id INT,
20     amount NUMERIC(10,2),
21     FOREIGN KEY(seller_id) REFERENCES SalesPerson(seller_id)
22 );
23
24 INSERT INTO SalesPerson VALUES
25 (1,'John','New York',15),
26 (2,'Amy','Los Angeles',13),
27 (3,'Mark','Chicago',12),
28 (4,'Pam','Boston',15);
29
30 INSERT INTO Company VALUES
31 (1,'RED','Boston'),
32 (2,'ORANGE','New York'),
33 (3,'YELLOW','Boston'),
34 (4,'GREEN','Austin');
35
36 INSERT INTO Orders VALUES
37 (1,1,1200),
38 (2,1,800),
39 (3,2,2500),
40 (4,3,1500),
41 (5,2,700),
42 (6,3,2000),
43 (7,4,3000),
44 (8,4,200);
```



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

Answer

```
1 SELECT seller_id
2
3 FROM Orders
4
5 GROUP BY seller_id
6
7 HAVING SUM(amount) =
8 (
9     SELECT SUM(amount)
10    FROM Orders
11   GROUP BY seller_id
12  ORDER BY SUM(amount) DESC
13   LIMIT 1
14 );
```

Output

seller_id

3



Question 3 – Department Top Three Salaries (Hard)

A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the top three highest *distinct* salary values within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee.

Given Data

```

1 CREATE TABLE Department
2 (
3     id     INT PRIMARY KEY,
4     name  VARCHAR(50)
5 );
6
7 CREATE TABLE Employee
8 (
9     id             INT PRIMARY KEY,
10    name           VARCHAR(50),
11    salary          INT,
12    departmentid   INT,
13    FOREIGN KEY(departmentid) REFERENCES Department(id)
14 );
15
16 INSERT INTO Department VALUES
17 (1, 'IT'),
18 (2, 'Sales');
19
20 INSERT INTO Employee VALUES
21 (1, 'Joe', 85000, 1),
22 (2, 'Henry', 80000, 2),
23 (3, 'Sam', 60000, 2),
24 (4, 'Max', 90000, 1),
25 (5, 'Janet', 69000, 1),
26 (6, 'Randy', 85000, 1),
27 (7, 'Will', 70000, 1);

```

Answer

```

1 SELECT
2 d.name    AS department,
3 r.name    AS employee,
4 r.salary  AS salary
5
6 FROM
7 (
8     SELECT
9         name,
10        salary,
11        departmentid,
12        DENSE_RANK() OVER (PARTITION BY departmentid ORDER BY salary DESC
13                             ) AS salary_rank
14     FROM Employee
15 ) r

```



```

15
16 JOIN Department d
17 ON r.departmentid = d.id
18
19 WHERE r.salary_rank <= 3
20
21 ORDER BY d.name, r.salary DESC;

```

Output

department	employee	salary
IT	Max	90000
IT	Joe	85000
IT	Randy	85000
IT	Will	70000
Sales	Henry	80000
Sales	Sam	60000

Home Task

For additional practice on the concepts covered in this experiment, the following problems are recommended:

1. <https://datalemur.com/questions/ad-campaign-roas>
2. <https://datalemur.com/questions/final-account-balance>

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood how to design reporting queries using **GROUP BY** together with correlated aggregate subqueries.
- Applied **UPDATE ... FROM** to modify records based on grouped aggregate values.
- Used **HAVING** with a subquery to correctly identify the maximum-valued group(s), including ties.
- Implemented the **DENSE_RANK()** window function to rank salaries within a partition.
- Learned to filter ranked results using a nested subquery to retrieve top-N distinct values per group.
- Gained practical exposure to solving real-world business reporting problems (sales and payroll analysis) using PostgreSQL.